Ingeniería y Calidad de Software

Segundo parcial · Unidades 2, 3 y 4

Resumen detallado

Unidad 2. Gestión Lean Agile de productos de software

1. Escalar Scrum: Nexus

1.1. El problema de trabajar con varios equipos

Escalar Scrum significa coordinar varios equipos sobre un mismo producto, preservando transparencia, inspección y adaptación. Más personas también generan dependencias: cambios en interfaces, datos compartidos y decisiones incompatibles. El progreso debe verse en el producto integrado.

Nexus es un framework de Scrum.org para aproximadamente 3 a 9 Scrum Teams. Comparten producto, Product Owner y Product Backlog. Su foco es reducir dependencias y problemas de integración. El Nexus Integration Team reúne al Product Owner, un Scrum Master y miembros apropiados; responde por lograr un incremento integrado terminado al menos una vez por Sprint. Orienta y facilita la integración, que sigue involucrando a los equipos.

1.2. Coordinación, eventos y artefactos

El refinamiento entre equipos identifica dependencias. Nexus Sprint Planning coordina objetivos y trabajo; Nexus Daily Scrum inspecciona integración y progreso. Nexus Sprint Review examina el incremento integrado con interesados y reemplaza las reviews individuales. Nexus Sprint Retrospective aborda mejoras entre equipos, complementada por sus retrospectivas.

El Product Backlog se compromete con el Product Goal; Nexus Sprint Backlog hace visibles trabajo y dependencias, con el Nexus Sprint Goal; el incremento integrado se compromete con una Definition of Done compartida.

1.3. Ejemplo: sistema de reservas

Supongamos que tres equipos trabajan en búsqueda, pagos y confirmaciones. Si búsqueda cambia el identificador de reserva y pagos sigue usando el anterior, ambos módulos podrían superar sus pruebas aisladas y fallar al conectarse. Antes de implementar, los equipos acuerdan el contrato y preparan una prueba integrada: buscar disponibilidad, reservar, pagar y recibir confirmación.

Una tarjeta terminada por un equipo todavía puede requerir integración y pruebas para formar parte del incremento integrado. Conviene dividir el trabajo en funcionalidades verificables, integrar cambios pequeños y resolver dependencias durante el Sprint. Acumular integración para el último día oculta problemas hasta que queda poco margen para corregirlos.

Este tema no tiene una presentación específica entre los archivos adjuntos. Se completa con la guía oficial de Nexus de Scrum.org: https://www.scrum.org/resources/online-nexus-guide.

2. Filosofía Lean aplicada al software

2.1. Valor y desperdicio

Lean busca maximizar el valor para el cliente y eliminar actividades que consumen recursos sin aportar valor. En software importa cuánto tarda una necesidad en convertirse en una solución utilizable, con qué calidad y con cuántas esperas o retrabajos.

Una persona ocupada todo el día no garantiza un producto que avance. Si hay diez funcionalidades empezadas y ninguna disponible, existe mucho trabajo en progreso y poco valor entregado. El enfoque Lean observa el flujo completo y cuestiona las optimizaciones locales que perjudican al conjunto.

2.2. Los siete principios de las diapositivas

- Eliminar desperdicios: reconocer trabajo innecesario, demoras y defectos, y actuar sobre sus causas.
- Amplificar el aprendizaje: usar entregas pequeñas, experimentos, pruebas y feedback para reducir incertidumbre.
- Diferir compromisos: conservar alternativas hasta disponer de información suficiente; decidir en el último momento responsable, antes de que la demora perjudique al producto.
- Entregar lo antes posible: reducir lotes y tiempo de espera para obtener valor y aprendizaje temprano.
- Dar poder al equipo: permitir que quienes conocen el trabajo participen en las decisiones y se organicen para resolverlo.
- Embeber la integridad conceptual: construir un producto coherente en su diseño, comportamiento y experiencia de uso; incorporar calidad durante el desarrollo.
- Ver el todo: mejorar el sistema completo, desde la necesidad del cliente hasta su satisfacción, evitando trasladar problemas entre áreas.

2.3. Los siete desperdicios más el talento no utilizado

- Trabajo parcialmente terminado: código sin integrar, historias pendientes de pruebas o análisis que todavía no se convierten en entregas. Inmoviliza esfuerzo y puede quedar obsoleto.
- Características extra: funcionalidades que no responden a una necesidad real. Cuestan implementación, pruebas, mantenimiento y complejidad.
- Proceso extra: aprobaciones duplicadas o documentación sin un uso concreto. Un control útil no es desperdicio por ser formal: se evalúa su aporte.
- Transferencias de conocimiento: pases entre áreas con pérdida de contexto, aclaraciones y repetición de explicaciones.
- Esperas: por decisiones, accesos, ambientes, revisiones o aprobaciones. El ítem sigue tardando aunque nadie lo esté trabajando.
- Cambios de tareas o de contexto: alternar entre muchas actividades obliga a reconstruir el contexto y aumenta errores y demoras.

- Defectos: generan investigación, corrección, nuevas pruebas y posibles incidentes en producción.
- Talento no utilizado: desaprovechar conocimientos y propuestas de las personas por una organización demasiado rígida.

La presentación distingue desperdicio tipo 1, necesario bajo las condiciones actuales, y tipo 2, puro desperdicio. El primero se minimiza y se busca cómo dejar de necesitarlo; el segundo se evita. Por ejemplo, una carga manual exigida por un sistema externo puede ser necesaria hoy, aunque convenga automatizarla en el futuro.

3. Método Kanban

3.1. Qué es y cómo introduce cambios

Kanban permite definir, gestionar y mejorar servicios de trabajo del conocimiento. Comienza con el proceso actual y promueve cambios evolutivos. Un tablero es una herramienta para visualizarlo, pero el método incluye límites, políticas, medición y mejora.

Sus valores son transparencia, equilibrio, colaboración, foco en el cliente, flujo, liderazgo, entendimiento, acuerdo y respeto. Los principios de gestión del cambio proponen comenzar con lo que se hace ahora, respetar inicialmente roles y responsabilidades, acordar una mejora evolutiva y fomentar liderazgo en todos los niveles.

Los principios de entrega de servicios agregan comprender necesidades del cliente, gestionar el trabajo permitiendo autoorganización de las personas y revisar periódicamente servicios y políticas. Se mejora a partir de la realidad del servicio, evitando imponer cambios sin comprenderla.

3.2. Prácticas generales

- Visualizar el trabajo: mostrar estados, tarjetas, responsables, bloqueos y colas. Todos deben comprender la misma situación.
- Limitar el WIP: establecer cuántos ítems pueden estar en progreso en cada parte del flujo.
- Gestionar el flujo: observar demoras, bloqueos y cuellos de botella para terminar trabajo de manera predecible y sostenible.
- Hacer explícitas las políticas: acordar reglas visibles de entrada, salida, prioridad y tratamiento de excepciones.
- Implementar ciclos de feedback: revisar el trabajo y el servicio con cadencias apropiadas.
- Mejorar colaborativamente y evolucionar mediante experimentos: formular una hipótesis, probar un cambio y evaluar resultados antes de consolidarlo.

3.3. WIP y sistema pull

WIP significa trabajo en progreso. Un límite de WIP evita iniciar indefinidamente tareas nuevas mientras otras permanecen abiertas. El sistema pull permite tomar un nuevo ítem cuando existe capacidad disponible en la etapa siguiente. En un sistema push se envía trabajo sin considerar si quien lo recibe puede procesarlo.

Ejemplo: Desarrollo tiene límite 3 y Testing límite 2. Si las dos posiciones de Testing están ocupadas, producir más código no elimina el cuello de botella. Conviene colaborar para terminar

pruebas, corregir defectos o resolver bloqueos. El límite provoca una conversación sobre el sistema, en lugar de ocultar el problema detrás de una cola creciente.

Las columnas de “hecho” de una actividad pueden funcionar como colas para la siguiente y deben quedar incluidas en las políticas de WIP correspondientes. Un elemento bloqueado sigue existiendo dentro del sistema: debe visualizarse y tratarse según una política explícita, evitando esconderlo para aparentar capacidad libre.

La ocupación del 100 % deja poca holgura para atender variaciones e imprevistos. Por eso, maximizar ocupación individual puede aumentar el tiempo total de entrega. El objetivo es terminar trabajo y mantener el flujo.

3.4. Políticas y clases de servicio

Una política debe ser simple, visible, acordada, aplicable y modificable. Define cuándo reponer trabajo, quién decide, qué significa terminar una actividad, los límites WIP, la prioridad y las cadencias. Ejemplo: una tarjeta entra a Testing cuando tiene build disponible, criterios de aceptación y datos de prueba preparados.

- Expreso: trabajo con costo de demora muy alto. En el ejemplo de clase tiene WIP 1 y puede interrumpir otros trabajos o exceder límites bajo una excepción acordada. Si todo es urgente, deja de existir una prioridad útil.
- Fecha fija: necesita una fecha concreta. Ingresa cuando corresponde para cumplirla y respeta el WIP; si la fecha queda en riesgo puede cambiar de clase según las políticas.
- Estándar: se ordena por valor de negocio y puede utilizar FIFO cuando no hay prioridades superiores.
- Intangible: el costo de demora es menos visible a corto plazo, como ciertas mejoras preventivas. Necesita una política que evite postergarlo indefinidamente.

Tipos de trabajo y clases de servicio son dimensiones distintas. Un defecto puede ser estándar o expreso según su impacto. También puede distribuirse capacidad entre desarrollo, mantenimiento y defectos; los porcentajes 60/30/10 de la presentación son un ejemplo, no una regla general.

3.5. Aplicación y mejora

Primero se modela el proceso real y se elige la unidad de trabajo: historia, defecto, solicitud o mejora. Luego se visualiza el flujo, se fijan límites WIP y políticas de calidad, se identifican cuellos de botella y se establecen cadencias para reposición, entregas y revisión.

Ejemplo de experimento: “Si reducimos los ítems iniciados y ayudamos diariamente a resolver bloqueos en Testing, bajará el tiempo de ciclo”. Se observa el resultado durante un período, se compara con datos previos y se decide mantener o modificar el cambio. Agregar columnas por sí solo no garantiza una mejora.

4. Métricas en los enfoques tradicional, Agile y Lean

4.1. Para qué medir

Una métrica transforma datos en información para responder una pregunta y decidir. Debe tener definición, unidad, forma de obtención y período de observación. Las diapositivas recomiendan simplicidad: medir sólo lo que aporta información útil y comparar para comprender, evitando juzgar personas mediante números aislados.

Se distinguen métricas de proyecto, de proceso y de producto. Las de proyecto permiten controlar un trabajo particular; al consolidarlas pueden ofrecer información del proceso organizacional. Las de producto describen propiedades de lo construido, como tamaño o defectos.

4.2. Enfoque tradicional: plan frente a ejecución

Las métricas básicas son tamaño, esfuerzo, tiempo calendario, costos y defectos. El esfuerzo mide trabajo humano, por ejemplo horas-persona; la duración mide tiempo transcurrido. Dos personas trabajando 4 horas simultáneas representan 8 horas-persona y 4 horas de duración, si no hay otras esperas.

A nivel del desarrollador pueden observarse esfuerzo estimado y real, cobertura unitaria y defectos detectados en pruebas o revisiones. A nivel del equipo: tamaño, hitos, dotación, tareas completadas, distribución del esfuerzo, cambios de requerimientos y defectos de integración. A nivel organizacional: precisión de estimaciones, presupuesto, esfuerzo y defectos de las releases.

Ejemplo: se estiman 100 horas-persona y se consumen 120. El desvío de esfuerzo respecto de la estimación es (120 - 100) / 100 × 100 = 20 %. Esto requiere analizar causas: cambios de alcance, estimación deficiente o retrabajo. El porcentaje no explica por sí mismo el problema.

4.3. Enfoque Agile: software funcionando y feedback

La prioridad es entregar software valioso, probado y funcionando. La medición debería surgir del trabajo, con la mínima carga necesaria.

- Velocidad: cantidad de trabajo terminado por un equipo en una iteración, habitualmente en puntos de historia. Sólo se cuenta lo que cumple la definición de terminado. Sirve para pronosticar con datos del mismo equipo; los puntos no son una unidad uniforme entre equipos.
- Capacidad: disponibilidad del equipo durante el Sprint, considerando personas, días, ausencias y otras obligaciones. Puede expresarse en horas disponibles. Informa cuánto trabajo puede asumirse; no equivale a la velocidad obtenida.
- Running Tested Features (RTF): funcionalidades ejecutables y probadas que realmente están disponibles. Refleja avance integrado y no simplemente tareas o código escritos.

Ejemplo: las velocidades de tres Sprints son 28, 32 y 30 puntos. El promedio de 30 orienta una previsión si se mantienen condiciones semejantes; no obliga a comprometer 30 siempre. Si en el próximo Sprint hay vacaciones, la menor capacidad debe considerarse. Una historia “90 % terminada” que no satisface la DoD no se cuenta como terminada.

4.4. Enfoque Lean: tiempos y eficiencia del flujo

- Lead time: desde que se solicita un ítem hasta su entrega. Representa la espera desde la perspectiva del cliente.

- Cycle time: desde el inicio del procesamiento hasta su finalización, según los límites definidos. Refleja el desempeño interno del proceso e incluye las esperas que ocurran dentro de ese intervalo.
- Touch time: tiempo efectivamente dedicado a trabajar sobre el ítem; excluye colas y bloqueos sin trabajo activo.
- Eficiencia del ciclo: touch time / tiempo total transcurrido × 100. Debe explicitarse si el denominador elegido es el lead time o el cycle time.

Ejemplo con días de trabajo: una solicitud llega el día 0, comienza el día 3 y se entrega el día 11. Lead time = 11 días; cycle time = 8 días. Si hubo 2 días de trabajo efectivo, la eficiencia sobre el ciclo es 2/8 × 100 = 25 %, y sobre el recorrido completo es 2/11 × 100 ≈ 18,2 %. Las demás horas o días corresponden a esperas u otros tiempos sin trabajo sobre ese ítem.

No deben mezclarse días hábiles y calendario en un mismo cálculo. Tampoco debe interpretarse cycle time como tiempo activo: un ítem puede permanecer una semana en el proceso y haber sido trabajado sólo unas horas.

Las métricas orientadas al servicio distinguen expectativa del cliente, capacidad real del sistema, acuerdo con el cliente y umbral por debajo del cual el servicio es inaceptable. Prometer tiempos exige conocer el comportamiento del servicio, no sólo expresar un deseo.

4.5. Métricas del producto y lectura conjunta

El tamaño puede describirse mediante líneas de código, requerimientos, casos de uso, funcionalidades o clases. Los puntos de historia representan una estimación relativa del trabajo de un equipo, por lo que no deben tratarse como una medida física universal del producto.

La densidad de defectos relaciona defectos con una unidad de tamaño: 12 defectos en 6.000 líneas equivalen a 2 defectos por mil líneas. La cobertura expresa qué proporción de un universo definido fue ejercitada: requisitos, sentencias, decisiones, etc. Una cobertura alta no prueba que las verificaciones sean correctas ni que el producto satisfaga al usuario.

Los enfoques se complementan: el tradicional pregunta por cumplimiento del plan y recursos; Agile por funcionalidad terminada y capacidad de adaptación; Lean por demoras y flujo. Ninguna métrica aislada mide toda la calidad ni todo el valor.

Unidad 3. Gestión de software como producto

5. Prácticas continuas

5.1. Continuidad y gestión del producto

Gestionar un producto implica sostener su evolución, mantenimiento y calidad después de una entrega. Las prácticas continuas acortan el intervalo entre cambio, verificación y feedback. El propósito es evitar grandes lotes de trabajo sin validar y mantener el producto en condiciones de evolucionar.

En los adjuntos, este eje aparece especialmente mediante TDD, refactoring, automatización y retrospectivas. Integración, entrega y despliegue continuos se explican aquí como complemento para cubrir el tema solicitado; no hay una presentación específica de esas prácticas entre los archivos recibidos.

5.2. Integración, entrega y despliegue continuos

Integración continua (CI) consiste en incorporar cambios con frecuencia a la línea principal y verificarlos mediante una construcción y pruebas automatizadas. Requiere control de versiones, una construcción repetible y atender enseguida las fallas. Así disminuye el riesgo de descubrir tarde cambios incompatibles.

Entrega continua mantiene el producto preparado para pasar a producción cuando se decide hacerlo. Extiende la verificación hacia un proceso de entrega repetible. Despliegue continuo lleva automáticamente a producción los cambios que superan los controles: la diferencia es esa decisión automática de liberar.

Ejemplo de pipeline: cambio en Git, compilación, pruebas unitarias, integración y preparación de una versión candidata. Con entrega continua puede quedar esperando una decisión de negocio; con despliegue continuo pasa automáticamente a producción al aprobar los controles definidos. Una pipeline automática con integraciones muy espaciadas no basta para practicar CI.

Complemento conceptual basado en Martin Fowler: https://martinfowler.com/articles/continuousIntegration.html y https://martinfowler.com/bliki/ContinuousDelivery.html.

5.3. Desarrollo guiado por pruebas: TDD

TDD es una técnica de desarrollo y diseño que combina escribir pruebas primero y refactorizar. Antes de implementar se expresa el comportamiento esperado mediante una prueba, habitualmente unitaria. La prueba obliga a pensar cómo se usará el componente, qué entrada recibe y qué resultado debe producir.

El ciclo es Red-Green-Refactor:

- Red: escribir una prueba pequeña y comprobar que falla por la razón esperada. Puede ser necesario crear la estructura mínima para compilar; luego se busca observar una falla de comportamiento.
- Green: escribir sólo el código de producción suficiente para que la prueba pase y ejecutar la suite pertinente.

- Refactor: mejorar estructura, nombres y duplicaciones sin cambiar el comportamiento observable; las pruebas deben seguir pasando.

Las tres leyes presentadas son no escribir código de producción sin una prueba fallando, no escribir más prueba que la necesaria para fallar y no escribir más producción que la necesaria para pasar. Promueven pasos pequeños y feedback inmediato.

5.4. Ejemplo de TDD y triangulación

Primera prueba: sumar(1, 2) debe devolver 3. Como todavía no existe el comportamiento, falla. Una implementación mínima podría devolver 3 de forma constante: es Fake It, una solución temporal que permite avanzar de rojo a verde.

Segunda prueba: sumar(3, 2) debe devolver 5. La constante deja de servir. Se generaliza la implementación a devolver a + b y se ejecutan ambas pruebas. La triangulación utiliza ejemplos diferentes para impulsar una solución más general. Después puede mejorarse el nombre de un método o quitar duplicación sin cambiar los resultados.

Pasar únicamente la primera prueba no demuestra que la suma esté bien implementada para todos los casos. El ciclo se repite con casos relevantes, incluidos negativos, cero o límites según el dominio. TDD guía el diseño, pero no reemplaza pruebas de integración, aceptación, seguridad o rendimiento.

El caso Taxi Mobile del material aplica la misma idea a mostrar taxis por barrio y estado: se prepara un usuario autorizado, una ciudad y taxis conocidos; se solicita el mapa y se comprueba que el resultado respeta los filtros. Para evitar una prueba engañosa conviene incluir taxis que deben aparecer y otros que deben quedar excluidos. Comprobar únicamente “todos los devueltos son libres” podría pasar con una lista vacía; también debe verificarse el conjunto esperado.

5.5. Patrones de TDD

Patrones para iniciar o ampliar pruebas: Starter Test comienza con un caso sencillo; One Step Test incorpora un paso mínimo; Explanation Test ayuda a aclarar la expectativa; Learning Test explora una API externa; Another Test agrega una variante; Regression Test reproduce un defecto para evitar que reaparezca.

Patrones para llegar a verde: Fake It usa una solución provisional; triangulación agrega ejemplos que fuerzan generalización; Obvious Implementation aplica una solución evidente; One to Many permite evolucionar desde un elemento hacia varios.

Patrones de organización: Isolated Test evita dependencias entre pruebas; Test List mantiene una lista de casos pendientes; Test First conserva el orden prueba-implementación; Assert First comienza por expresar la comprobación; Test Data y Evident Data buscan datos legibles y una relación clara entre entradas y resultados.

Los patrones xUnit del segundo material incluyen assertion, que comprueba una condición; fixture, que prepara el contexto; test method, que contiene un caso; exception test, que comprueba errores esperados; y suites que agrupan pruebas. Su propósito es favorecer pruebas repetibles, comprensibles y mantenibles.

5.6. Refactoring como práctica continua

Refactorizar es mejorar la estructura interna conservando el comportamiento observable. Ejemplos: renombrar para expresar intención, extraer un método, eliminar duplicación o separar responsabilidades. Cambiar una regla de negocio exige pruebas del nuevo comportamiento y no es una refactorización por sí mismo.

La secuencia de clase propone verificar que las pruebas pasan, identificar un código que “huele mal”, decidir cómo simplificar, aplicar cambios pequeños y ejecutar las pruebas otra vez. Así se incorpora lo aprendido y se evita el deterioro del diseño.

Los beneficios esperados de TDD y refactoring son código más comprensible, cohesivo, menos acoplado y más fácil de cambiar. Sus límites incluyen el aprendizaje inicial, el mantenimiento de pruebas y la dificultad de actuar sobre código muy acoplado. Tener pruebas no garantiza cobertura completa ni ausencia de defectos.

5.7. Retrospectivas y mejora continua

Una retrospectiva permite al equipo examinar un período de trabajo para mejorar el siguiente. Puede realizarse al finalizar una iteración o release. Busca comprender hechos, identificar causas y acordar acciones sobre cómo se trabaja. La Sprint Review inspecciona resultados del producto con interesados; la retrospectiva inspecciona la forma de trabajo del equipo.

Las cinco etapas de la presentación son:

- Preparar el escenario: explicitar objetivo, agenda y acuerdos para que todos participen.
- Reunir datos: construir una imagen compartida de hechos, dificultades y resultados; evitar decidir sólo por impresiones.
- Generar ideas: interpretar patrones y causas, y proponer alternativas.
- Decidir qué hacer: priorizar pocas mejoras, asignar responsables y definir resultados observables. Para iteraciones cortas, el material recomienda no más de dos o tres acciones.
- Cerrar: revisar compromisos, agradecer y evaluar la propia retrospectiva.

Ejemplo: los datos muestran cinco historias esperando ambiente. La causa es una preparación manual que depende de una sola persona. Se acuerda automatizarla, con responsable y fecha, y medir si disminuyen las esperas en la iteración siguiente. “Comunicarnos mejor” es demasiado impreciso para verificar su cumplimiento.

Señales de una retrospectiva ineficaz: sólo hablan líderes, se evita el problema central, no se prioriza, faltan datos o las mejoras anteriores nunca se implementan. Se revisan procesos e interacciones, sin convertir la reunión en una evaluación de desempeño individual. Las retrospectivas necesitan seguimiento y no sustituyen la excelencia técnica ni la capacitación.

Unidad 4. Aseguramiento de calidad de proceso y de producto

6. Conceptos generales de calidad

6.1. Qué significa calidad

La calidad reúne características de un producto o servicio que permiten satisfacer necesidades explícitas e implícitas. En software involucra expectativas del cliente, usuario, gerencia, equipo de desarrollo y mantenimiento, y demás interesados. Es relativa al contexto: un atributo importante para una aplicación puede ser secundario para otra.

Las perspectivas de clase son: trascendental, calidad reconocida como excelencia; usuario, adecuación a sus necesidades; manufactura, conformidad con especificaciones y procesos; producto, atributos medibles; y valor, relación entre beneficios y costos. Un sistema puede cumplir su especificación y resultar poco útil si ésta no representa la necesidad real.

6.2. Calidad de producto, proceso y proyecto

La calidad del producto se refiere a sus propiedades: funcionalidad, seguridad, fiabilidad o mantenibilidad. La calidad del proceso considera cómo se desarrolla y mantiene: actividades, responsabilidades, estándares y controles. La gestión del proyecto agrega planificación, recursos, riesgos y seguimiento que hacen posible el trabajo.

Un proceso de software combina actividades, métodos, prácticas y transformaciones realizadas para producir o mantener software y productos asociados. Definirlo exige aclarar quién participa, qué hace, cuándo y cómo. Interactúa con personas y tecnología; un proceso bien definido mejora previsibilidad y aprendizaje, aunque no garantiza por sí solo un producto correcto.

6.3. Principios, ventajas y dificultades

La calidad se construye durante todo el ciclo de vida; no se agrega al final mediante más pruebas. Requiere responsabilidad compartida, capacitación, apoyo gerencial, mediciones útiles, liderazgo con el ejemplo y planificación. Debe ser razonable para el negocio y empezar por prácticas simples.

Ventajas: detectar problemas temprano reduce retrabajo; los estándares hacen más consistente el trabajo; los controles ofrecen visibilidad; un producto confiable favorece satisfacción y mantenimiento; el análisis de causas evita repetir fallas. El costo de corregir suele crecer cuando el defecto atraviesa más etapas y afecta más productos de trabajo.

Dificultades y costos: formación, herramientas, reuniones, preparación de evidencia y tiempo de evaluación. Los beneficios pueden aparecer después de adquirir experiencia. Una implantación rígida puede generar burocracia o resistencia. La respuesta es adaptar las prácticas al riesgo y al contexto, manteniendo controles útiles. Eliminar controles para ganar velocidad puede trasladar el costo a producción.

7. Administración y aseguramiento de la calidad

7.1. Aseguramiento, planificación y control

Aseguramiento de calidad (SQA) establece y evalúa procesos, estándares y procedimientos para dar confianza sobre la forma de trabajo. Planificación de calidad selecciona y adapta los que se aplicarán al proyecto, define atributos deseados y cómo evaluarlos. Control de calidad verifica resultados y cumplimiento mediante revisiones, pruebas y mediciones.

Las diapositivas también incluyen dentro del control la comprobación de que se respetan procedimientos. Por eso no conviene separar las actividades sólo por su nombre: debe mirarse su objetivo. Una revisión puede aportar información sobre un producto y sobre el proceso que lo produjo.

Ejemplo: establecer que los cambios deben revisarse pertenece al sistema de aseguramiento; decidir responsables y criterios de revisión para este proyecto es planificación; revisar un cambio y registrar los defectos es una actividad concreta de control.

7.2. Actividades de SQA y plan de calidad

SQA evalúa planificación del proyecto, requerimientos, diseño, programación, integración y pruebas, y prácticas de seguimiento. Desarrolla o facilita métodos, herramientas, técnicas y estándares para revisiones; adapta procedimientos al proyecto y comunica incumplimientos para su resolución.

Un plan de calidad identifica los productos y atributos críticos, estándares aplicables, actividades de evaluación, responsables, recursos, calendario, registros y tratamiento de desviaciones. El PACS, Plan de Aseguramiento de Calidad de Software, organiza las actividades de aseguramiento y las auditorías previstas.

Los estándares de producto definen características del resultado, como convenciones de código o estructura documental; los de proceso definen cómo realizar actividades, como una inspección o el control de cambios. Pueden ser internacionales, nacionales, organizacionales o propios del proyecto.

El Grupo de Aseguramiento de Calidad (GAC) necesita suficiente independencia para informar problemas sin quedar subordinado a la presión del proyecto. El material recomienda que no reporte al gerente de proyecto y que tenga acceso cercano a la alta gerencia. Esto permite escalar desviaciones; no lo convierte en el único responsable de la calidad.

7.3. Mejora de procesos e IDEAL

El circuito de mejora consiste en definir procesos, desarrollar el producto, evaluar resultados, identificar mejoras y estandarizar las prácticas que funcionan. Si un estándar es inadecuado, debe revisarse; cumplirlo mecánicamente no aporta calidad.

IDEAL organiza la mejora en iniciar, diagnosticar, establecer, actuar y aprender. Se obtiene apoyo y contexto, se caracteriza la práctica actual, se priorizan acciones, se prueban e implantan cambios y se analizan lecciones para el siguiente ciclo. En el material aparecen las denominaciones Learning y una referencia a Leveraging; la idea central es aprovechar el aprendizaje para sostener la mejora.

8. Modelos y normas de calidad: CMMI, SPICE e ISO

8.1. CMMI: capacidad y madurez

CMMI significa Capability Maturity Model Integration. Es un modelo de referencia para mejorar y evaluar procesos. La presentación desarrolla principalmente CMMI-DEV v1.3, sus áreas de proceso y representaciones históricas; estos conceptos se conservan aquí para estudiar el material, sin presentar sus versiones como actuales.

La representación por etapas establece madurez organizacional mediante conjuntos de áreas. La continua analiza capacidad de áreas de proceso y permite priorizar mejoras específicas. Madurez organizacional y capacidad de un proceso son conceptos relacionados, pero no equivalentes.

Los cinco niveles por etapas son:

- 1. Inicial: procesos poco predecibles, reactivos y dependientes de esfuerzos individuales.
- 2. Administrado: los proyectos planifican y controlan su trabajo; se gestionan requerimientos, compromisos y productos.
- 3. Definido: se utiliza un proceso organizacional establecido y adaptado a cada proyecto; hay una base compartida de prácticas y capacitación.
- 4. Administrado cuantitativamente: se gestionan desempeño y variabilidad mediante objetivos y métodos cuantitativos.
- 5. Optimizado: se actúa sistemáticamente sobre causas y oportunidades para mejorar el desempeño.

En nivel 2, las diapositivas ubican administración de requerimientos, planificación, monitoreo y control, acuerdos con proveedores, medición y análisis, PPQA y administración de configuración. En nivel 3 agregan prácticas de ingeniería, riesgos, formación, definición y mejora de procesos. En niveles 4 y 5 aparecen gestión cuantitativa, desempeño organizacional y análisis causal.

Las constelaciones históricas presentadas son DEV para desarrollo, SVC para servicios y ACQ para adquisición. CMMI se evalúa mediante un appraisal; el material enfatiza que no es una norma ISO ni una certificación del producto. La comparación con Agile debe entenderse como enfoques complementables: disciplina de proceso y entrega adaptable pueden convivir si se implementan según el contexto.

8.2. SPICE y evaluación de procesos

SPICE se asocia a ISO/IEC 15504 y a la evaluación de capacidad y mejora de procesos. Se examinan procesos concretos y evidencia para identificar brechas y orientar acciones. El marco contempla evaluación, modelos de referencia y medición de capacidad; su evolución se relaciona con la familia ISO/IEC 330xx.

La escala clásica de capacidad distingue 0 incompleto, 1 realizado, 2 gestionado, 3 establecido, 4 predecible y 5 en optimización. El foco es cuánto se logra el propósito de un proceso y cómo se gestiona, establece y mejora. Una organización puede presentar capacidades distintas entre procesos; un buen resultado en pruebas no demuestra igual capacidad en gestión de requerimientos.

La presentación menciona SPICE de forma breve. Esta explicación conceptual complementa ese punto; ISO describe su marco de evaluación en https://www.iso.org/standard/37454.html.

8.3. Qué cubren las normas ISO

ISO no designa un único modelo. ISO 9001 se refiere a un sistema de gestión de calidad; ISO/IEC 12207 trata procesos del ciclo de vida del software; ISO/IEC 15504 y la familia 330xx se relacionan con evaluación de procesos; la familia ISO/IEC 25000, SQuaRE, aborda requisitos y evaluación de calidad del producto. ISO/IEC 25012 se centra en calidad de datos.

Una certificación de un sistema de gestión no implica que cada producto esté libre de defectos. Debe diferenciarse evaluar cómo trabaja la organización de medir propiedades del software concreto.

8.4. Calidad del producto según el esquema de clase

Las diapositivas usan ISO/IEC 25010:2011. Se conserva esa clasificación de ocho características para mantener coherencia con la clase:

- Adecuación funcional: completitud de las funciones, corrección de resultados y pertinencia para las tareas del usuario.
- Eficiencia de desempeño: tiempos de respuesta, uso de recursos y capacidad.
- Compatibilidad: coexistencia e interoperabilidad con otros sistemas.
- Usabilidad: reconocimiento de adecuación, aprendizaje, operación, protección ante errores, estética y accesibilidad.
- Fiabilidad: madurez, disponibilidad, tolerancia a fallos y recuperación.
- Seguridad: confidencialidad, integridad, autenticidad, responsabilidad y no repudio.
- Mantenibilidad: modularidad, reutilización, analizabilidad, modificación y facilidad de prueba.
- Portabilidad: adaptación, instalación y reemplazo en distintos entornos.

Ejemplo: “permite registrar un pago” es funcional; “responde en menos de dos segundos bajo la carga acordada” es desempeño; “un usuario no puede ver pagos ajenos” es seguridad. Para evaluar calidad, el atributo general debe convertirse en una condición verificable.

La calidad en uso considera lo que ocurre cuando personas concretas utilizan el sistema en un contexto; no equivale solamente a inspeccionar código. La calidad de datos considera la información utilizada. Un cálculo correcto con datos incorrectos puede producir un resultado inútil.

La edición 2011 usada por la cátedra fue reemplazada por modelos posteriores. ISO/IEC 25010:2023 presenta nueve características de producto; no se mezclan aquí ambas clasificaciones: https://www.iso.org/standard/78176.html.

9. Auditorías de software

9.1. Objetivo y tipos

Una auditoría es una evaluación independiente de productos o procesos contra criterios objetivos, como estándares, especificaciones y procedimientos. Busca evidencia de cumplimiento, da visibilidad gerencial y descubre riesgos y oportunidades de mejora. No se limita a preguntar si alguien cree que hizo bien su trabajo.

La auditoría de proyecto verifica que se siga el proceso definido y que exista consistencia entre productos a medida que evolucionan. Contrasta requerimientos con diseño, código y planificación

de verificación y validación. ERS significa Especificación de Requerimientos de Software; DDS, Documento de Diseño de Software.

La auditoría de configuración funcional compara el software y la evidencia disponible con la ERS: busca que los requerimientos y capacidades especificados estén implementados. Revisa también que la trazabilidad esté actualizada. La auditoría de configuración física compara el producto construido con la documentación técnica y los elementos que se entregan: versiones, componentes y descripción deben corresponderse.

Ejemplo: comprobar que una release permite anular una reserva exigida por la ERS es foco funcional. Comprobar que el ejecutable, el código identificado y el manual corresponden a la misma release es foco físico. Comprobar que los cambios se aprobaron y revisaron conforme al proceso es foco de auditoría de proyecto.

9.2. Auditoría al Grupo de Calidad

El temario pide además auditar al propio Grupo de Calidad. Aunque las diapositivas de auditorías desarrollan principalmente proyecto y configuración, el mismo principio de independencia se aplica al GAC: también debe demostrarse que su proceso de aseguramiento funciona.

Se revisa si planifica controles, dispone de recursos y competencias, ejecuta auditorías y revisiones previstas, registra resultados, informa desviaciones, realiza seguimiento y escala problemas sin resolver. Las evidencias incluyen PACS, calendario, informes, registros de hallazgos, planes de acción y comprobantes de cierre. Quien evalúa necesita independencia respecto de las actividades examinadas.

Ejemplo: el GAC emitió diez informes, pero no verificó ninguna corrección. La cantidad de informes demuestra actividad documental, mientras que la falta de seguimiento puede revelar un incumplimiento del proceso de aseguramiento. Auditar al grupo evita asumir que controlar a otros lo exime de ser controlado.

9.3. Responsabilidades

El gerente de SQA prepara el plan de auditorías, estima costos, asigna recursos y asegura la gestión de no conformidades. El auditor prepara y conduce la evaluación, solicita y contrasta evidencia, registra hallazgos, informa y verifica acciones. El auditado acuerda condiciones, participa, aporta evidencia, responde al informe, propone acciones y comunica su cumplimiento.

El auditor necesita escuchar, preguntar con claridad, tomar notas y evitar una actitud acusatoria. Se recomiendan preguntas abiertas iniciales —qué, cómo, quién, cuándo— y preguntas cerradas para precisar. La evidencia debe sostener las conclusiones, incluso si el auditado está nervioso o aporta información muy extensa.

10. Proceso de auditoría

10.1. Preparación y planificación

Se acuerdan alcance, objetivo, criterios, lugar, fecha y participantes. El auditor puede solicitar documentación preliminar, revisarla y preparar un checklist. El auditado reúne información y evidencia. El alcance define qué entra en la auditoría; el criterio determina contra qué se evaluará.

Ejemplo de criterio: “todo requerimiento aprobado debe identificarse unívocamente y contar con trazabilidad a sus pruebas”. La pregunta derivada es cómo se registra esa relación; la evidencia puede ser una matriz, casos versionados y resultados de ejecución.

10.2. Ejecución

La reunión de apertura presenta participantes, confirma alcance y cronograma, explica el proceso y aclara dudas. Luego se realizan entrevistas, revisión de registros, muestreo y verificaciones asistidas por herramientas. Se completa el checklist y se conservan referencias a la evidencia.

Un checklist incluye identificación del proyecto, fase, fecha, auditor, auditados y roles, objetivo, alcance y preguntas. Guía la auditoría sin reemplazar el análisis. Preguntar “¿hay plan?” es insuficiente: también interesa si está actualizado, asigna responsables y recursos, fue comunicado y sus cambios se aprobaron.

10.3. Análisis y reporte

El auditor revisa notas, checklist y documentos, contrasta las desviaciones contra el criterio y prepara resultados. En la reunión de cierre explica hallazgos y aclara dudas; luego emite el informe final con identificación, participantes, alcance, resultados, comentarios y solicitudes de planes de acción.

La presentación distingue buenas prácticas, desviaciones y observaciones. Una buena práctica supera de manera destacable lo esperado. Una desviación incumple un proceso o requisito y requiere plan de acción. Una observación señala una condición mejorable o un riesgo de futura desviación; según el esquema de clase no exige el mismo plan formal.

Un hallazgo útil vincula criterio, evidencia y diferencia encontrada. “Falta calidad” no es verificable. “Tres de ocho cambios muestreados carecen de la aprobación exigida por el procedimiento” sí describe una desviación concreta.

10.4. Seguimiento y cierre

El auditado analiza las desviaciones y presenta acciones, responsables y fechas. El auditor revisa el plan, sigue su ejecución y verifica cumplimiento antes de cerrar. Cuando corresponda, debe comprobarse que la acción ataca la causa y no sólo completa un documento faltante.

Corregir es resolver el caso detectado; una acción correctiva apunta a su causa para evitar repetición. Si falta una aprobación, obtenerla resuelve ese caso; revisar por qué el flujo permitió omitirla ayuda a prevenir nuevos casos. El cierre se apoya en evidencia.

Métricas sugeridas por la presentación: esfuerzo y duración por auditoría, cantidad de desviaciones y clasificación por área de proceso. Más hallazgos no significa automáticamente peor trabajo: influyen el alcance, riesgo y profundidad del muestreo.

11. Calidad de producto: fundamentos del testing

11.1. Qué es probar software

Testing comprende planificación, preparación, evaluación y pruebas de productos de software y productos de trabajo asociados. Busca detectar diferencias con lo requerido, evaluar características y aportar información sobre adecuación al uso. Las diapositivas enfatizan una actitud crítica: diseñar pruebas capaces de revelar defectos.

El testing muestra problemas y reduce incertidumbre, pero no garantiza por sí solo calidad ni ausencia de defectos. La prevención también depende de requerimientos claros, diseño, revisiones, prácticas de programación y mejora del proceso. Corregir las causas de defectos repetidos tiene un efecto que ejecutar más pruebas sin cambiar nada no logra.

11.2. Error, defecto, falla, severidad y prioridad

Una distinción de uso habitual es: error, equivocación humana; defecto, problema introducido en un producto de trabajo; falla, comportamiento incorrecto observado al ejecutar. Ejemplo: una persona interpreta mal la regla, programa “mayor que 18” cuando correspondía “mayor o igual”, y el sistema rechaza a un usuario de 18 años.

El material de revisiones utiliza una clasificación propia según origen y gravedad del hallazgo. Al estudiar cada técnica, importa reconocer qué terminología usa, conservando la relación entre causa humana, problema en el artefacto y efecto observable.

Severidad indica impacto técnico o de negocio: bloqueante, crítico, mayor, menor o cosmético en la escala presentada. Prioridad indica urgencia de resolución. Un error visual en la página de una campaña puede ser poco severo y tener prioridad alta por una fecha próxima; un problema grave en una función todavía no liberada puede tener otra prioridad temporal.

11.3. Verificación y validación

Verificar pregunta si se está construyendo correctamente el producto respecto de sus especificaciones. Validar pregunta si se construye el producto correcto para las necesidades reales. Son actividades de todo el ciclo de vida: pueden empezar revisando requerimientos y continuar con diseño, código y pruebas.

Ejemplo: un cálculo implementa exactamente una fórmula escrita, pero esa fórmula interpreta mal la política comercial. Puede superar la verificación contra esa especificación y fallar en validación. Revisiones estáticas y pruebas dinámicas son aproximaciones complementarias; no son sinónimos exclusivos de verificar y validar.

11.4. Siete principios del testing

- Muestra presencia de defectos: superar pruebas no demuestra que no existan otros.
- El testing exhaustivo es imposible en sistemas reales: se prioriza por riesgos y recursos.
- Testing temprano: revisar y diseñar pruebas desde etapas iniciales permite descubrir problemas antes.
- Agrupamiento de defectos: ciertos componentes concentran problemas; los datos ayudan a orientar el esfuerzo.
- Paradoja del pesticida: repetir siempre las mismas pruebas deja de descubrir defectos nuevos; deben revisarse y ampliarse.
- Dependencia del contexto: un sistema crítico necesita una estrategia diferente de una aplicación simple.
- Falacia de ausencia de errores: un sistema con pocos defectos detectados puede fracasar si no resuelve la necesidad del usuario.

La independencia ayuda a detectar supuestos que el autor pasa por alto. Esto complementa las pruebas de los desarrolladores: no significa que deban dejar de probar su código ni que testers y

programadores sean adversarios.

12. Planificación y proceso de pruebas

12.1. Qué debe resolver el plan

La planificación define objetivos, riesgos, alcance y exclusiones; niveles y tipos de prueba; técnicas; recursos y responsabilidades; ambientes y datos; calendario; automatización; criterios de entrada y salida; y mecanismos de registro y reporte. Debe relacionarse con requerimientos y necesidades de los interesados.

Los criterios de aceptación del producto describen condiciones para aceptar una funcionalidad. Los criterios de salida de una actividad de prueba permiten decidir cuándo darla por completada. Las diapositivas también usan “criterio de aceptación” para este cierre; conviene distinguir ambos usos por su contexto.

Ejemplo de cierre: todos los casos críticos ejecutados, ninguna falla bloqueante abierta y riesgos residuales comunicados y aceptados por quien corresponda. Un porcentaje alto de pruebas aprobadas no basta si la prueba que falla corresponde al cobro de pagos.

12.2. Etapas del proceso

- Planificación y control: comprender objetivos y riesgos, construir el plan, comparar avance y resultados, y ajustar decisiones.
- Identificación, análisis y especificación: revisar la base de pruebas, evaluar testeabilidad, identificar condiciones y datos, diseñar y priorizar casos, y preparar el entorno.
- Preparación y ejecución: implementar casos y automatizaciones, verificar el ambiente, ejecutar sobre una versión identificada, registrar resultados y comparar con lo esperado.
- Evaluación, reporte y cierre: comprobar criterios, comunicar resultados y riesgos, verificar entregables y correcciones, consolidar datos y analizar lecciones aprendidas.

El proceso puede repetirse sobre nuevas versiones. Un ciclo de pruebas aplica el conjunto definido de casos a una versión. Registrar versión, configuración y datos permite reproducir fallas y evitar comparar resultados de productos diferentes como si fueran el mismo.

12.3. Caso de prueba y reporte de defecto

Un caso contiene objetivo, precondiciones y ambiente, datos de entrada, pasos y resultado esperado. La presentación resume tres partes esenciales: objetivo; datos y ambiente; comportamiento esperado. Un resultado esperado concreto permite decidir si la ejecución cumple.

Ejemplo: objetivo, rechazar una reserva sin plazas; precondición, viaje con cero plazas disponibles; entrada, solicitud de una plaza; acción, confirmar reserva; esperado, mensaje de falta de disponibilidad, ninguna reserva creada y plazas sin modificación. También se registra resultado real y estado de ejecución.

Un reporte de defecto útil agrega título preciso, pasos reproducibles, versión, entorno, esperado frente a observado, severidad y evidencia. El objetivo es que otra persona pueda reproducir e investigar el problema sin reconstruir todas las condiciones desde cero.

12.4. Ambientes, regresión y decisión de detenerse

Se distinguen desarrollo, prueba, preproducción y producción. Preproducción debería asemejarse suficientemente al entorno real en configuración, dependencias y condiciones pertinentes. Una diferencia de ambiente puede ocultar fallas que no aparecen en la máquina del desarrollador.

Confirmación o retest verifica que el defecto corregido ya no se reproduce. Regresión busca efectos secundarios sobre funcionalidades que antes funcionaban. Después de arreglar un descuento se repite el caso que fallaba y se comprueba que no se alteraron impuestos, totales o facturación relacionados.

El smoke test es una verificación rápida y amplia de operaciones esenciales que determina si una construcción es suficientemente estable para pruebas más profundas. No reemplaza una suite completa. Se decide cuánto probar según riesgo, costo y criterios acordados; no por la suposición de que no habrá defectos.

13. Niveles y tipos de pruebas

13.1. Los cuatro niveles

Pruebas unitarias: ejercitan componentes pequeños de manera relativamente aislada. Suelen realizarlas desarrolladores con acceso al código, frameworks y dobles de prueba. Ejemplo: comprobar que calcularTotal devuelve el importe correcto para cantidades conocidas.

Pruebas de integración: verifican interacción entre componentes que individualmente pueden funcionar. Revisan contratos, formatos, secuencias y acceso a servicios o datos. Conviene integrar incrementalmente y atender temprano componentes críticos. Top-down comienza por módulos superiores y puede usar stubs para los faltantes; bottom-up comienza por inferiores y puede usar drivers para simular llamadas superiores.

Pruebas de sistema: evalúan la aplicación integrada como un todo, con requerimientos funcionales y no funcionales y un ambiente representativo. Ejemplo: efectuar el recorrido completo de reserva y pago, incluyendo persistencia, notificaciones y recuperación ante una interrupción.

Pruebas de aceptación: generan confianza sobre adecuación a necesidades de usuarios y negocio. Su foco principal es la aceptabilidad del sistema, aunque puedan descubrir defectos. La presentación incluye alfa en un entorno controlado y beta en condiciones reales de uso.

Nivel indica sobre qué alcance se prueba; tipo indica qué característica se evalúa. Una prueba funcional puede existir en varios niveles. Una prueba unitaria no se define exclusivamente por usar caja blanca.

13.2. Funcionales y no funcionales

Las funcionales comprueban qué hace el software, basándose en requerimientos, reglas y procesos del negocio. Ejemplo: una anulación devuelve la plaza y respeta la política de reintegro.

Las no funcionales evalúan cómo funciona bajo condiciones definidas. Necesitan criterios medibles: “rápido”, “seguro” o “fácil” son insuficientes sin precisar condiciones y expectativas.

- Rendimiento: tiempos de respuesta, concurrencia y recursos consumidos.
- Carga: comportamiento ante una carga esperada o especificada.

- Estrés: comportamiento al superar capacidad o ante recursos restringidos; interesa también la recuperación.
- Volumen: comportamiento con grandes cantidades de datos.
- Seguridad: acceso, permisos, protección de información y operaciones no autorizadas.
- Usabilidad: facilidad de aprendizaje y operación, claridad y prevención de errores del usuario.
- Fiabilidad y recuperación: continuidad, tolerancia y restablecimiento ante fallos.
- Portabilidad, configuración e instalación: funcionamiento en entornos, plataformas y combinaciones de dependencias, y correcta instalación o actualización.
- Mantenibilidad: facilidad de analizar, modificar y probar; parte de su evaluación puede realizarse mediante revisiones y análisis estático.

Ejemplo: simular 300 usuarios concurrentes esperados corresponde a carga; aumentar hasta provocar degradación permite explorar estrés; cargar millones de reservas históricas evalúa volumen. No son pruebas idénticas aunque puedan combinarse.

14. Técnicas y herramientas para probar

14.1. Caja negra: especificaciones y comportamiento

Caja negra diseña pruebas a partir de entradas, salidas y reglas observables sin depender de la estructura interna. Es útil para validar funcionalidad y detectar omisiones frente a especificaciones. Necesita un resultado esperado confiable.

Partición de equivalencias divide el dominio en clases cuyos elementos se espera que reciban un tratamiento semejante. Se identifican clases válidas e inválidas y se eligen representantes. Así se reduce el número de pruebas manteniendo una selección sistemática.

Ejemplo: un campo admite edades enteras de 18 a 65 inclusive. Clases: menor que 18, entre 18 y 65, mayor que 65; además, entradas no enteras o vacías si el sistema puede recibirlas. Representantes posibles: 10, 30, 80 y un texto. Una prueba por clase no demuestra que todos los valores sean correctos; es una estrategia de selección.

Análisis de valores límite concentra pruebas en los bordes, donde suelen aparecer errores de comparación. Para ese rango se prueban 17, 18, 19 y 64, 65, 66. Si se permiten decimales, deben definirse precisión y valores inmediatamente vecinos según el dominio.

14.2. Ejemplo de las franjas impositivas

La presentación propone ingresos hasta 4.000 sin impuesto, luego una franja de 1.500 al 10 %, otra de 2.000 al 22 % y montos superiores al 40 %, indicando porcentajes sobre el total. Se interpreta aquí como una regla de ejercicio de clase, no como legislación ni impuesto progresivo real.

Bajo esa lectura, las clases son ingreso hasta 4.000; mayor que 4.000 y hasta 5.500; mayor que 5.500 y hasta 7.500; y mayor que 7.500. Deben aclararse ingresos negativos, precisión y redondeo. Los límites críticos son 4.000, 5.500 y 7.500; conviene probar cada límite y sus vecinos válidos para la unidad monetaria admitida.

Ejemplo: con ingresos enteros, 5.000 corresponde a 10 % del total y da 500; 6.000 corresponde a 22 % y da 1.320. Si el negocio quisiera porcentajes marginales sobre cada tramo, los resultados

serían otros. Diseñar pruebas ayuda a descubrir esta ambigüedad antes de programar.

14.3. Técnicas basadas en experiencia

Conjetura o adivinanza de defectos usa experiencia para buscar fallas probables: campos vacíos, fechas imposibles, doble envío, cero, datos extremos, permisos incorrectos o interrupción de conexión. Complementa el diseño sistemático.

Testing exploratorio integra aprendizaje, diseño y ejecución durante la exploración. Puede organizarse con una misión, tiempo acotado, notas y evidencias. Por ejemplo: explorar qué ocurre con una reserva si el usuario retrocede, repite una confirmación o pierde conexión. No equivale a hacer clic sin propósito.

14.4. Caja blanca y cobertura

Caja blanca utiliza la estructura interna para diseñar casos. Cobertura de sentencias comprueba qué instrucciones se ejecutaron; cobertura de decisiones, qué resultados de cada decisión se recorrieron; cobertura de condiciones, qué valores tomaron las condiciones simples. Otros criterios combinan condiciones o consideran caminos.

Ejemplo: si se autoriza cuando “usuarioActivo Y saldoSuficiente”, una prueba con ambas verdaderas y otra con usuarioActivo falso pueden recorrer las salidas verdadera y falsa. Sin embargo, podrían no evaluar saldoSuficiente falso; por cortocircuito incluso puede no evaluarse la segunda condición. Distintos criterios de cobertura responden preguntas diferentes.

Cobertura = elementos ejecutados / elementos considerados × 100. Tener 100 % de sentencias no implica 100 % de decisiones ni ausencia de errores. Un requisito omitido no aparece automáticamente al recorrer el código existente.

14.5. Herramientas y selección

La presentación muestra JUnit para pruebas unitarias Java. Una assertion compara un resultado con lo esperado: assertEquals(9, suma.sumar()) falla si sumar no devuelve 9 para los valores preparados. Los métodos de preparación establecen contexto y los de limpieza liberan recursos; las anotaciones concretas dependen de la versión del framework.

Otras categorías útiles son gestores de casos y defectos, herramientas de automatización de API o interfaz, generación de carga, cobertura, análisis estático y servidores de integración continua. Se eligen según el riesgo y la tarea. Los ejemplos de categorías no implican que una marca o producto particular haya sido exigido en las diapositivas.

Ninguna técnica es completa: combinar especificaciones, límites, estructura y experiencia ofrece perspectivas complementarias. Automatizar conviene especialmente para comprobaciones frecuentes, repetibles y estables; deben considerarse costo de preparación, mantenimiento y confiabilidad de resultados.

15. Revisiones técnicas del software

15.1. Propósito y alcance

Una revisión técnica es una actividad estática: examina un producto de trabajo sin ejecutarlo. Puede aplicarse a requerimientos, diseño, código, planes y casos de prueba, incluso si están incompletos. Detecta defectos cerca de su origen y considera atributos como claridad, uniformidad y mantenibilidad.

Sus ventajas son detección temprana, menor propagación de problemas y aprendizaje compartido. Sus costos incluyen capacitación, guías, plantillas, herramientas, preparación individual y reuniones. Las inspecciones formales requieren experiencia y organización para que sus beneficios compensen el esfuerzo inicial.

El material distingue hallazgos mayores, con posible efecto operacional o incumplimiento de necesidades; menores, que perjudican claridad o mantenimiento sin producir necesariamente una falla operacional; y notas cosméticas, como errores tipográficos. La clasificación sirve para priorizar y medir; no toda observación constituye un defecto grave.

15.2. Walkthrough e inspección

Walkthrough o recorrida: el autor guía a participantes a través del producto; se formulan preguntas y comentarios. Suele tener poca preparación y formalidad, y sirve para comprender, compartir conocimiento y descubrir problemas. En la clasificación de la presentación no equivale a una revisión técnica formal.

Inspección: aplica un proceso planificado con preparación individual, roles, checklists, registro de hallazgos, corrección, seguimiento y mediciones. Busca detectar defectos de manera eficiente. La reunión se centra en identificar y registrar, sin intentar resolver extensamente cada problema ni evaluar a su autor.

La presentación también incluye una decisión sobre el estado del producto tras la revisión. Esto puede usarse para gestionar su avance, pero el propósito técnico central de la inspección sigue siendo detectar defectos y comprobar su tratamiento, no funcionar como una aprobación burocrática.

15.3. Roles de una inspección

- Autor: crea o mantiene el producto, lo entrega y realiza las correcciones; comunica retrabajo y resultados.
- Moderador: planifica, selecciona participantes con el autor, distribuye material, conduce la reunión y sigue los hallazgos.
- Lector: recorre o presenta el producto de forma sistemática durante la inspección.
- Anotador: registra hallazgos, decisiones y datos.
- Inspector: prepara su revisión individual, identifica defectos y registra esfuerzo.

Se eligen revisores según el producto: código necesita perspectiva técnica; requerimientos también requieren conocimiento del negocio; un plan de pruebas se beneficia de testers, desarrollo, arquitectura y calidad. Tener muchos asistentes sin preparación no sustituye un equipo apropiado.

15.4. Proceso de inspección

Planificación: elegir producto y alcance, participantes, materiales y calendario. Visión general: explicar contexto y proceso. Preparación: cada inspector examina el material con criterios y guías. Reunión: reunir hallazgos, aclararlos y descartar falsos positivos. Corrección: el autor trata los defectos. Seguimiento: verificar cambios, decidir si hace falta reinspección y consolidar datos.

El informe responde qué se revisó, quiénes participaron, qué se encontró y cuál es el resultado. Las diapositivas sugieren distribuir material con anticipación y limitar la reunión a dos horas. Las tasas y tamaños de ejemplo sirven como orientación para planificar; no son una velocidad universal exigible.

15.5. Técnicas, herramientas y métricas

Se utilizan listas de revisión, guías de lectura, comparación con requisitos y estándares, trazabilidad, anotaciones y registro de hallazgos. Herramientas de revisión de código, diferencias entre versiones y análisis estático pueden apoyar el trabajo, pero no reemplazan la comprensión del dominio.

Ejemplo de checklist de requerimientos: ¿cada requisito es identificable, claro, consistente, completo y verificable? Para código: ¿se manejan límites y errores, hay duplicación, los nombres expresan intención y se respeta el contrato? Para pruebas: ¿el esperado es concreto, el ambiente es reproducible y se contemplan casos negativos?

Densidad de defectos = defectos / tamaño revisado. Esfuerzo de inspección = planificación + preparación + reunión + retrabajo, según la definición de clase. Esfuerzo por defecto = esfuerzo / defectos encontrados. Porcentaje de reinspecciones = reinspecciones / inspecciones × 100. Para porcentaje corregido se usa defectos corregidos / defectos detectados × 100; no debe confundirse con esfuerzo por unidad de tamaño, aunque una tabla de las diapositivas mezcle esas etiquetas.

Reglas de trabajo: revisar el producto, respetar agenda, limitar debates, registrar por escrito, exigir preparación y formación, y aprovechar experiencias anteriores. Una revisión hostil desalienta comunicar problemas y reduce su utilidad.

16. Testing en ambientes ágiles

16.1. Cambio de enfoque y manifiesto

El testing ágil integra evaluación y prevención durante todo el desarrollo, con feedback corto y responsabilidad compartida. El manifiesto presentado prioriza probar durante el trabajo, prevenir defectos, comprender lo que se evalúa, ayudar a construir un mejor sistema y hacer responsable de la calidad a todo el equipo.

La comparación con el enfoque secuencial destaca el riesgo de concentrar pruebas después de implementar. En Agile se aclaran expectativas, diseñan casos, desarrollan y prueban funcionalidades en cada iteración. Esto no elimina planificación ni documentación: adapta su volumen a lo necesario para trabajar y mantener evidencia útil.

16.2. Los nueve principios de la presentación

- Adelantar el testing en el proyecto.

- Tratarlo como actividad continua y no como fase separada.
- Involucrar a todo el equipo.
- Reducir la latencia del feedback.
- Usar pruebas para representar expectativas.
- Mantener código limpio y corregir defectos rápidamente.
- Reducir documentación innecesaria de pruebas.
- Incluir pruebas en la definición de terminado.
- Evolucionar desde probar al final hacia desarrollo conducido por pruebas.

Las seis prácticas concretas son automatización unitaria y de integración, TDD, regresión automatizada a nivel sistema, ATDD, pruebas exploratorias y versionado de pruebas junto con el código. Se complementan: ninguna cubre todos los riesgos por sí misma.

16.3. ATDD y criterios de aceptación

ATDD significa desarrollo conducido por pruebas de aceptación. Antes de implementar, representantes de negocio, desarrollo y testing acuerdan ejemplos que expresan expectativas. Esos ejemplos orientan el desarrollo y pueden automatizarse como comprobaciones de aceptación.

Ejemplo: dado un viaje con una plaza disponible y un usuario habilitado, cuando confirma una reserva, entonces se crea exactamente una reserva y la disponibilidad pasa a cero. Otro ejemplo debe establecer qué ocurre si dos usuarios intentan reservar esa última plaza simultáneamente.

TDD guía diseño de unidades y componentes; ATDD clarifica comportamiento esperado desde negocio. Pueden coexistir. Criterios de aceptación se refieren a una historia concreta; la DoD establece condiciones compartidas de calidad del incremento, como integración, pruebas y ausencia de problemas incompatibles con su entrega.

16.4. Rol del tester y cultura del equipo

El tester ayuda al Product Owner a aclarar historias y criterios, a desarrolladores a identificar riesgos y casos límite, y al equipo a preparar datos, ambientes, exploraciones y automatización. También interpreta resultados, comunica riesgos y colabora con mediciones. Su trabajo exige pensamiento crítico, comunicación y conocimientos técnicos y de negocio.

Responsabilidad compartida no significa que desaparezca la especialidad. El tester aporta una mirada distinta, pero la calidad no puede delegarse completamente. Desarrolladores deben construir y probar; negocio debe aclarar expectativas; el equipo debe actuar frente a defectos.

Los desafíos culturales incluyen pérdida de identidad del rol, temor a responsabilidades nuevas, falta de formación y expectativas irreales sobre automatización. La transición requiere acompañamiento, aprendizaje y acuerdos claros. El feedback sobre el producto debe comunicarse sin culpar a las personas.

16.5. Los cuatro cuadrantes del testing ágil

Los cuadrantes combinan orientación al negocio o tecnología, y propósito de apoyar al equipo o criticar/evaluar el producto. No son etapas obligatorias ni una secuencia temporal.

Q1, tecnología y apoyo al equipo: pruebas unitarias y de componentes, generalmente automatizadas. Ayudan a construir correctamente pequeñas piezas y a modificar con feedback rápido.

Q2, negocio y apoyo al equipo: ejemplos, pruebas funcionales y de aceptación de historias, prototipos y simulaciones. Aclaran qué debe construirse; pueden combinar revisión humana y automatización.

Q3, negocio y evaluación del producto: exploratorias, escenarios, usabilidad y aceptación del usuario. Investigan utilidad y comportamiento desde la experiencia real; tienen una participación humana importante.

Q4, tecnología y evaluación del producto: rendimiento, carga, estrés, seguridad, recuperación y otros atributos técnicos. Suelen requerir herramientas y entornos especializados.

Ejemplo aplicado: una reserva usa Q1 para calcular el precio; Q2 para acordar la política de cancelación; Q3 para descubrir si el usuario entiende el reintegro; Q4 para estudiar pagos concurrentes y recuperación ante una caída.

16.6. Pirámide y automatización

La pirámide propone una base amplia de pruebas unitarias o de componentes, una capa intermedia de servicios/API y reglas de negocio, y menos pruebas completas a través de interfaz gráfica. Las de bajo nivel suelen ser rápidas y precisas para localizar problemas; las de interfaz cubren recorridos relevantes, pero suelen resultar más lentas y frágiles.

Las diapositivas contrastan esta forma con una pirámide invertida dominada por pruebas GUI. Sus porcentajes son ilustrativos: la distribución concreta depende de arquitectura y riesgos. Las pruebas manuales y exploratorias complementan las capas automatizadas y no se eliminan al adoptar la pirámide.

Los beneficios de automatización mencionados son rapidez, reutilización, precisión y mayor cobertura y alcance, con posible reducción de costos. Exige mantener scripts, datos y ambientes; una prueba inestable o con una comprobación incorrecta puede dar falsa confianza. Automatizar muchas interacciones no asegura verificar bien el resultado.

16.7. Calidad integrada en una historia

Antes de implementar se acuerdan criterios y ejemplos. Durante el trabajo se utilizan TDD cuando corresponde, revisión técnica, integración frecuente y pruebas unitarias, de integración y aceptación. Se realizan exploraciones y evaluaciones no funcionales según riesgos. La historia sólo se considera terminada cuando satisface sus criterios y la DoD.

La review permite contrastar el incremento con necesidades y obtener feedback de producto. La retrospectiva utiliza lo aprendido para mejorar el trabajo siguiente. Así se conectan calidad de producto, calidad de proceso y mejora continua dentro del alcance de las unidades del parcial.